

DL-CAT: 带解耦学习选择器的 CAT 框架 精读笔记

原文: s40747-023-01019-1.pdf (Complex & Intelligent Systems, 2023, 9:5555–5566) · 代码: 未发现开源代码 · 笔记于 2026-06-27

一、解决什么问题

这篇论文做的是 CAT (计算机化自适应测试) 里的「选題器」。CAT 系统由两个模块循环交替组成:

1. 学生能力估计模块 (用认知诊断模型 / IRT 类模型): 根据学生已经答过的题和对错, 估计出当前能力 $\hat{\theta}$ 。
2. 选題器 (question selector) π : 根据当前的 $\hat{\theta}$ 和题目特征, 从题库里挑下一道最该考的题。

目标是: 用尽量少的题, 把学生真实能力 θ^0 估准。

痛点 (已有方法的毛病):

- 启发式选題器 (MFI 最大费雪信息量、KLI、MAAT 等): 靠人手设计的固定准则 (比如“挑信息量最大的题”), 有先验偏好, 抓不住数据本身的复杂规律; 而且很多是“模型相关”的——换个能力估计模型就得重新设计准则, 尤其难给神经网络型的能力估计模型配选題器。
- 学习型选題器 (BOBCAT、NCAT, 把 CAT 写成双层优化 bilevel optimization): 性能不错, 但两个模块的参数学习是耦合的——外层优化的结果用来衡量内层优化的好坏, 反之亦然。后果是: 题库一加新题, 整个选題器就得从头重训, 在真实教育场景里很不实用。

本文的卖点 (DL-CAT): 把选題器和能力估计模型彻底解耦。- 用一个深度网络 (MLP) 直接预测“每道候选题的选择分数”, 不靠固定准则。- 选題器训练时完全不依赖能力估计模型怎么训练——这就是“解耦 (decoupled)”。- 好处: 加新题不用重训选題器 (只需重训能力估计模型拿到新题的难度/区分度参数, 直接喂给现成的选題器即可)。这是它相对 BOBCAT 最实在的优势, 效率实验里体现得很明显。

最关键的技术难点是: 选題“选得好不好”没有现成标签 (no ground truth)。普通监督学习有标签可学, 但“这道题是不是该选的最优题”没人告诉你。论文的核心贡献就是人工造一个近似真值来当标签, 再配一个成对排序损失来学。

二、问题定义 (输入 / 输出)

符号: - 学生集 $S = \{s_1, \dots, s_L\}$, 题库 $E = \{e_1, \dots, e_M\}$ 。- 作答日志: 三元组 (s, e, r) , $r \in \{0, 1\}$, 1= 答对、0= 答错。- 能力估计模型 $\mathcal{M}$ (这里用 IRT/MIRT), 选題器 π 。- 学生 s_i 选完第 $k-1$ 题后的能力估计 $\hat{\theta}_i^{(k-1)}$; 真实能力 θ_i^0 (通常未知)。- 题目 e_j 的难度 β_j 、区分度 α_j 。

选題器要学的东西 (输入 $\rightarrow$ 输出): - 输入: 当前估计的学生能力 $\hat{\theta}_i^{(k-1)}$ + 候选题 e_j 的参数 (α_j, β_j) , 拼成一个向量 $X_{ij} = [\theta_i, \alpha_j, \beta_j]$ 。- 输出: 一个分数 p_{ij} (学生 s_i 该被选到题 e_j 的得分)。对题库里所有候选题各算一个分, 挑分最高的那道作为下一题。

整个 CAT 流程的输入输出 (一个小例子): 假设学生小明 s_i , 题库有 5 道题。第 1 步系统初始化 $\hat{\theta}_i^{(0)} = 0$, 选題器给 5 道题打分, 挑出分最高的 e_3 让小明答; 小明答对了 ($r = 1$) $\rightarrow$ 能力估计模型把 $\hat{\theta}_i$ 更新为 0.4; 第 2 步再用 $\hat{\theta}_i^{(1)} = 0.4$ 重新打分挑下一题.....重复 n 步后输出题序列 $E_i = \{e_3, e_1, \dots\}$, 并希望此时的 $\hat{\theta}_i^{(n)}$ 已经很接近小明真实能力 θ_i^0 。

注意: 能力 $\hat{\theta}_i$ 的更新只靠“答过哪些题、对错如何”, 跟选題器本身无关——这正是解耦的体现。

三、模型结构 (四步流水线)

对应论文图 2, DL-CAT 训练分四步:

1. 预训练认知诊断模型 $\mathcal{M}_C$ (如 IRT): 在全体学生作答日志上用交叉熵训练, 训完后从里面抽出参数——学生能力 θ_i 、题目难度 β_j 和区分度 α_j 。这步给后面提供“原材料”。
2. 选題器打分: 把 $[\theta_i, \alpha_j, \beta_j]$ 拼起来喂进一个 3 层 MLP, 输出每道候选题的分数 p_{ij} 。
3. 真值构造模块造标签: 因为没有现成标签, 就用“如果真选了这道题, 能力估计会变得多准”来反推标签——具体是算一个 RMSE (在元集上验证)。RMSE 越小, 说明这题选得越好。
4. 算损失、更新选題器: 拿选題器的预测分 P_i 和真值模块给的 RMSE 序列, 用成对排序损失更新 MLP 参数。

对所有学生重复, 直到选題器收敛。推理时直接用 MLP 给候选题打分、取最高分即可。

为什么这么设计? 关键是第 3 步。它不去问“这题客观上是不是最优” (无法知道), 而是用一个模拟评估——把题选了、更新能力、在留出的元集上看预测误差 RMSE——把“选題质量”翻译成可计算的数。这样选題器就有了可学习的监督信号, 且这个信号的产生不需要把选題器塞进能力模型的训练里, 于是两者解耦。

四、关键公式详解

公式 (1): IRT 作答概率

$$P_k(\theta_i) = \sigma[\alpha_k(\theta_i - \beta_k)]$$

- θ_i : 学生能力; β_k : 题目难度; α_k : 区分度; σ : sigmoid.
- 举例: 小明能力 $\theta_i = 1.0$, 题 e_k 难度 $\beta_k = 0.5$ 、区分度 $\alpha_k = 2$ 。则 $\alpha_k(\theta_i - \beta_k) = 2 \times (1.0 - 0.5) = 1.0$, $P = \sigma(1.0) = \frac{1}{1+e^{-1}} \approx 0.73$ 。即小明有 73% 概率答对。能力恰好等于难度 ($\theta = \beta$) 时概率正好 0.5; 区分度 α 越大, 曲线越陡, 能力稍微高于难度就明显答得对。
- 作用: CAT 里能力估计的根基; 也是后面选器输入里 α, β 的来源。

公式 (2): 费雪信息量 (MFI 选题准则)

$$I_k(\hat{\theta}_i^{(n)}) = \frac{P'_k(\hat{\theta}_i^{(n)})}{P_k(\hat{\theta}_i^{(n)}) [1 - P_k(\hat{\theta}_i^{(n)})]}$$

- 对 IRT 可简化为 $I_k = \alpha_k^2 P_k(1 - P_k)$ 。
- 举例: 接上例 $P_k = 0.73$, $\alpha_k = 2$, 则 $I_k = 2^2 \times 0.73 \times 0.27 \approx 0.79$ 。若另一题 $P = 0.5$ (难度正好卡在能力上)、同样 $\alpha = 2$, $I = 4 \times 0.5 \times 0.5 = 1.0$ 更大——所以 MFI 倾向于挑“答对概率 50%、即难度贴着当前能力估计、且区分度高”的题。
- 作用: 这是传统启发式选器的代表准则, 论文拿它当对比基线 (baseline)。DL-CAT 想用学习的方式“超越”这种固定准则。

公式 (3): 认知诊断模型的训练损失

$$\text{loss} = -\frac{1}{n} \sum_{(s_i, e_j, r_{ij}) \in D} r_{ij} \log \hat{r}_{ij} + (1 - r_{ij}) \log(1 - \hat{r}_{ij})$$

- 标准的二分类交叉熵, $\hat{r}_{ij} = \mathcal{M}_C(s_i, e_j)$ 是预测答对概率。
- 举例: 真实 $r = 1$ (答对)、预测 $\hat{r} = 0.73$, 单项损失 $= -\log 0.73 \approx 0.31$; 若预测 $\hat{r} = 0.2$ (模型以为会答错), 损失 $= -\log 0.2 \approx 1.61$, 惩罚大很多。最小化它就是让模型预测和真实对错一致。
- 作用: 第 1 步预训练 $\mathcal{M}_C$, 训完抽出 θ, α, β 给选器用。

公式 (4)(5)(6): 从嵌入到选器输入再到打分

(4) 是 IRT 的网络化前向: 学生 one-hot $\rightarrow$ 嵌入 $\rightarrow \theta_i$; 题目 one-hot $\rightarrow$ 嵌入 $\rightarrow \alpha_j, \beta_j$ 。

(5) 把三者拼成选器输入:

$$X_{ij} = [\theta_i, \alpha_j, \beta_j]$$

(6) 是选器 MLP:

$$p_{ij} = \sigma(W_h \cdot y_{h-1} + b_h)$$

- 举例: $\theta_i = 1.0, \alpha_j = 2, \beta_j = 0.5$, 输入向量 $[1.0, 2, 0.5]$ (单维 IRT 下是 3 维; MIRT 下 θ, α 是多维, 向量更长)。喂进隐藏层大小 8 的 3 层 MLP, 最后 sigmoid 输出比如 $p_{ij} = 0.62$ 。对题库每道题候选题各算一个 p , 取最大的那道选出去。
- 作用: 用神经网络替代固定准则, 让“选题打分”完全数据驱动。

公式 (7): 真值构造 (核心创新之一)

$$\text{Rmse}_{ij} = \sqrt{\frac{1}{|D_V^i|} \sum_{f=1}^{|D_V^i|} (r_{if} - \mathcal{M}(s_i, e_f | \hat{\theta}_i, \alpha_f, \beta_f))^2}$$

- 把学生数据分成支持集 D_T^i (用来选题、更新能力) 和元集 D_V^i (用来当“考卷”验证)。
- 流程: 先初始化 $\theta_i \rightarrow$ 在支持集里“选了题 e_j ”并据此更新得到 $\hat{\theta}_i \rightarrow$ 用这个 $\hat{\theta}_i$ 去预测元集 D_V^i 里每道题 e_f 的对错 $\rightarrow$ 和元集真实结果 r_{if} 比, 算 RMSE。
- 举例: 元集有 3 道题, 真实对错 $r = [1, 0, 1]$ 。若选了题 e_j 更新出的 $\hat{\theta}_i$ 预测元集为 $[0.8, 0.3, 0.7]$, 则 $\text{RMSE} = \sqrt{\frac{(1-0.8)^2 + (0-0.3)^2 + (1-0.7)^2}{3}} = \sqrt{\frac{0.04+0.09+0.09}{3}} = \sqrt{0.073} \approx 0.27$ 。换另一道题 $e_{j'}$ 若更新出的 $\hat{\theta}$ 预测更准、 $\text{RMSE}=0.15$, 说明 $e_{j'}$ 比 e_j 更值得选。
- 作用: 这就是给“选题质量”造出来的近似标签——**RMSE 越小, 题越好**。它让无监督的选题问题变成可学习。注意它用的是“选一题后能力估得准不准”来打分, 符合 CAT 终极目标 (估准能力)。

公式 (8): 成对排序损失 (核心创新之二)

$$L_i = \sum_{j=1}^q \sum_{k=1}^{q-1} \max(0, A(\text{rmse}_{ij}, \text{rmse}_{ik}) \cdot (p_{ij} - p_{ik}))$$

$$A = \begin{cases} -a, & \text{if } \text{rmse}_{ij} < \text{rmse}_{ik} \\ +a, & \text{otherwise} \end{cases}$$

- 动机: 公式 (7) 造出来的 RMSE 标签彼此数值差异很小, 直接拿数值回归学不出顺序。所以不学绝对值, 只学“两道题谁更该选”的相对顺序 (pairwise)。
- 怎么读: 对每一对题 (j, k) ,
 - 若 $\text{rmse}_{ij} < \text{rmse}_{ik}$ (题 j 更好), $A = -a$, 损失项 $= \max(0, -a(p_{ij} - p_{ik}))$ 。要让损失 $= 0$, 就需要 $p_{ij} - p_{ik} \geq 0$, 即逼着选器给更好的题 j 打更高分。
 - 若给反了 ($p_{ij} < p_{ik}$, 好题打了低分), $-a(p_{ij} - p_{ik}) > 0$ 产生正损失, 被惩罚。
- 举例: 题 j 的 $\text{rmse}=0.15$ (更好), 题 k 的 $\text{rmse}=0.27$ 。若选器打分 $p_{ij} = 0.4 < p_{ik} = 0.6$ (排错了), $a = 1$ 时损失项 $= \max(0, -1 \times (0.4 - 0.6)) = \max(0, 0.2) = 0.2$, 受罚 $\rightarrow$ 反向传播会推高 p_{ij} 、压低 p_{ik} 。若打分 $p_{ij} = 0.7 > p_{ik} = 0.5$ (排对了), $\max(0, -1 \times 0.2) = 0$, 无损失。
- 惩罚系数 a (RQ4 的消融对象): $a = 1$ 不看位置; $a = (q - j)$ 越靠前的位置权重越大; $a = k$ 看比较位置; $a = (q - j) * k$ 两者都看——实验显示 $a = (q - j) * k$ 最好, 说明“排在前面的题排错了”应该罚得更重。
- 作用: 把“造出来的、差异微小的 RMSE 标签”转成稳定可学的排序信号, 是 DL-CAT 能训起来的关键。

五、代码实现对照

未发现开源代码。全文只在 Data availability 给了 ASSISTments 数据集链接(<https://sites.google.com/site/assistmentsdata/home/2009-2010-assistment-data>), Math 是私有数据集, 没有提供 GitHub/GitLab 等代码仓库。

不过算法 1 (训练流程) 已经把实现讲得相当清楚, 自己复现的话核心是: - 预训练一个 IRT/MIRT 拿 θ, α, β ; - 每个学生数据切 50%/50% 成支持集 D_T / 元集 D_V ; - 对支持集每道题: 拼特征 $\rightarrow$ MLP 打分 $p_{ij} \rightarrow$ 模拟更新 $\hat{\theta} \rightarrow$ 在 D_V 上算 rmse ; - 收集 (P_i, RMSE_i) 用公式 (8) 算成对损失、更新 MLP (隐藏层 8、3 层、Adam、lr=0.001、只取 top 40% 候选题算 RMSE 省算力)。可参照 BOBCAT (arXiv:2108.07386) 的开源实现来搭能力估计和数据管线部分。

六、小结

实验结论: - 有效性 (RQ1/RQ2): 在 ASSIST 和 Math 上、用 IRT 或 MIRT, DL-CAT 的 AUC/ACC 都最好 (如 ASSIST 第 10 步 IRT-AUC 0.6751 vs BOBCAT 0.6690, 提升 $>0.7\%$); 步数越大优势越明显; 能力估计的 RMSE 收敛也最快最好。- 损失设计 (RQ3): 成对排序损失稳定优于 ListMLE / ListNet / SetRank。- 惩罚系数 (RQ4): $a = (q - j) * k$ (同时考虑位置高低和比较位置) 最好。- 效率 (RQ5, 最大亮点): 测试阶段 DL-CAT 10 步仅 2.31s, 远快于 MAAT(385s)、MAAT Cov(420s)、BOBCAT(20.4s); 新增 50 题时 DL-CAT 只需 18s, 而 BOBCAT 要 5.8 小时——这就是解耦带来的实际价值。

优点: - 选器与能力估计模型解耦, 加新题免重训, 贴合真实教育场景; - 用造标签 + 成对排序解决了“选题无监督信号”的根本难题; - 模型无关 (agnostic), IRT/MIRT 都能套。

局限 / 可改进: - “近似真值”依赖元集划分和 RMSE 模拟, 质量受数据稀疏影响; 只取 top 40% 候选算 RMSE 是为省算力的近似。- 有效性提升幅度其实不大 (AUC 多在小数点后第三位), 真正的硬优势是效率而非精度。- 学生能力初始化用的是简单初始化, 作者也在未来工作里提到想用元学习改进冷启动。- 只在两个数据集上验证, Math 私有不可复现。

和 KT/CAT 研究的关联: 这是一篇典型的“选器 (item selection)”论文, 思路把“主动学习 / learning-to-rank”引入 CAT, 和 BOBCAT (双层优化)、NCAT (强化学习) 属同一条学习型选器脉络, 主打“解耦 + 工程实用性”。做 CAT item selection 时可作为效率基线和“如何无监督造选器标签”的参考。